Lexical Search Is Hard to Beat for Small Offline Coding Agents

Seeded graph and embedding signals add little over BM25 for locating issue files in the competition's own code graphs.

Gemma 4 Developer Agent Paper Track writeup (updated with v3 result)

Abstract

Small coding agents that run offline on consumer hardware have to find the right files inside a context window of 32,768 tokens. The Gemma 4 Developer Agent Competition gives every agent three code-intelligence tools (a neighbor lookup, a subgraph lookup, and a symbol-similarity search) backed by a precomputed call graph and per-symbol embeddings. We ask a practical question that a prompt author must answer before the agent runs: which of these signals should the agent trust to reach the files that a correct patch edits? We build a localization benchmark from the 129 training tasks, using the files edited by each reference patch as labels, and compare six retrieval strategies: BM25 over the issue text, the symbol named in the issue ("seed"), a two-hop graph expansion from the seed, embedding similarity to the seed, and two rank-fusion hybrids. On the 124 tasks whose gold files exist in the graph, BM25 alone reaches a mean reciprocal rank (MRR) of 0.360 (95% CI 0.295 to 0.428). Adding the graph raises it to 0.386, but the paired difference of +0.026 has a 95% interval of -0.034 to +0.086, so we cannot claim a gain. Graph expansion does beat the bare seed (+0.049, interval +0.017 to +0.082), and embedding similarity is clearly worse than graph expansion (-0.100, interval -0.167 to -0.033). The best signal depends on the repository: BM25 leads on FastAPI (0.389 against 0.218 for the graph), while seeded methods lead on Rich (0.347 and 0.381 against 0.218). A simple per-issue rule that picks a signal matches the fixed fusion exactly, while an oracle that picks the best signal per task would reach 0.544, so headroom exists, but neither a simple rule nor a learned router captures it. Returning the full text of ten BM25 hits costs about 15,500 tokens, close to half the window. In an end-to-end check, an agent prompt that follows the resulting policy scored 0.01 on the public leaderboard against 0.05 for our baseline prompt, so we claim no end-to-end benefit. We release the scripts, per-task results, and the figure so that the study can be repeated in one command.

1. Introduction

Resolving a GitHub issue starts with localization: deciding which files and functions to read and edit. Recent agent systems treat this step as central. SWE-agent gives a language model a purpose-built interface for navigating and editing a repository (Yang et al., 2024). Agentless splits the work into localization, repair, and validation, and shows that a fixed pipeline can match more autonomous agents (Xia et al., 2024). LocAgent and RepoGraph index a repository as a graph so that structure, and not only text, guides the search (LocAgent, arXiv:2503.09089; Ouyang et al., 2025). These systems use large models and generous budgets.

The Gemma 4 Developer Agent Competition changes the setting. The agent is a configuration file for a small open model, scored with vLLM at a maximum length of 32,768 tokens, with no internet access. The organizers ship, for each repository snapshot, a code graph and an embedding for every symbol, and they expose three tools over them: a neighbor lookup, a subgraph lookup, and a

similarity search. A prompt author must decide how the agent should combine these tools with ordinary text search. The decision matters because every tool result consumes context that the model also needs for reading code and writing a patch.

This paper measures the quality and the token cost of each signal on the competition's own training tasks. Our contributions are a 124-task localization benchmark built only from competition data, a comparison of six strategies with bootstrap intervals and a per-repository breakdown, evidence that lexical search is a strong baseline that is not significantly improved in aggregate, a router and oracle analysis, a token-cost analysis against the 32,768-token budget, and open scripts that reproduce every number.

2. Related Work

SWE-bench introduced the task of resolving real GitHub issues in Python repositories and showed that retrieval quality limits the performance of language models (Jimenez et al., 2024). Agent systems then competed on how to explore a repository. SWE-agent argues that the design of the agent-computer interface matters as much as the model (Yang et al., 2024). Agentless uses a hierarchical localization step that narrows from files to classes to edit locations, and it combines language-model reasoning with classic information retrieval (Xia et al., 2024).

Graph-based localization is the closest line of work. LocAgent represents a repository as a directed heterogeneous graph and lets an agent traverse it, reporting strong file-level accuracy on SWE-bench Lite (arXiv:2503.09089). RepoGraph adds a line-level dependency graph as a plug-in to existing frameworks (Ouyang et al., 2025). CoSIL searches a call graph at function level without a prebuilt index (arXiv:2503.22424). These papers evaluate large models and report end-to-end gains. Our study differs in three ways: it isolates retrieval from the model, it uses the exact graph and embedding artifacts that a small offline agent receives, and it reports confidence intervals for the contribution of each signal.

For fusion we use reciprocal rank fusion, which sums 1/(k + rank) over the input rankings and has been shown to beat individual systems and Condorcet fusion on several TREC collections (Cormack et al., 2009). For lexical scoring we use the BM25 family of ranking functions (Robertson and Zaragoza, 2009).

3. Data and Method

3.1 Tasks and labels

The competition provides 129 training tasks. Each task has a repository, a base commit, an issue text, and a reference patch. The repositories are FastAPI (67 tasks), Rich (48), requests (13), and httpx (1). The label set for a task is the set of Python files that the reference patch modifies, excluding test files. We keep a task only if at least one label file exists in the graph at the base commit. Five tasks are excluded because their patches only add new files, which leaves 124 tasks (FastAPI 64, Rich 46, requests 13, httpx 1).

3.2 Code graph and embeddings

For each repository snapshot the competition provides a graph and an embedding file. The graph is a multigraph whose nodes are modules, classes, and functions named by dotted paths, with the source text attached, and whose edges are typed relations such as calls. A FastAPI snapshot contains about 4,260 nodes. Each node has a 256-dimensional embedding. The harness documentation states that the sandbox has no live embedding model, so the similarity tool resolves a query against symbol names already in the graph and compares stored vectors. A free-text issue therefore cannot be embedded by the agent. This is why every embedding method in our study starts from a seed symbol.

3.3 Mapping nodes to files

We map nodes to files by module path. For each label file we take the longest dotted suffix of its path that matches at least one node name, and a node belongs to the file if its name equals that module or starts with it followed by a dot. Nodes that match no label file are grouped by their module prefix, so that the ranked list contains distinct files and distinct non-label modules.

3.4 Seeds

A seed is a graph node whose full name, or whose short name when it is unique enough (at most three nodes share it), appears verbatim in the issue text. This is a simple stand-in for what an agent would pass to the graph tools after reading the issue. Of the 124 tasks, 90 have at least one seed, and 44 have a seed inside a module that the patch edits. Methods that need a seed return an empty list for the other 34 tasks, and we report results both on all 124 tasks and on the 90 seeded tasks.

3.5 Retrieval strategies

- **lexical_bm25**: BM25 over each node's name and source text, with the issue text as the query. Identifiers are split on underscores and capitalization.
- **seed_only**: the seed nodes themselves.
- **graph_2hop**: nodes within two hops of any seed in the undirected graph, ordered by hop distance, then by degree, then by name.
- **embedding_seed**: nodes ranked by cosine similarity to the mean embedding of the seeds.
- **hybrid_rrf**: reciprocal rank fusion (k = 60) of BM25, graph, and embedding rankings.
- **hybrid_bm25_graph**: reciprocal rank fusion of BM25 and graph rankings.

3.6 Metrics and statistics

Nodes are converted to a ranked list of distinct files. We report MRR of the first label file, file recall at k (the fraction of label files in the top k, for k = 1, 5, 10, 25), and the estimated token cost of the top ten nodes (characters of source text divided by four, plus a fixed overhead per node). Confidence intervals use 10,000 bootstrap resamples over tasks with a fixed seed. Paired differences use the same resamples for both methods. Every run is deterministic, and we verified that two runs with different hash seeds give identical tables.

4. Results

4.1 Overall ranking quality

Table 1 shows results on all 124 tasks.

Method	MRR	95% CI	R@1	R@5	R@10	R@25
lexical_bm25	0.360	0.295 to 0.428	0.184	0.395	0.539	0.780
seed_only	0.254	0.187 to 0.325	0.158	0.276	0.305	0.306
graph_2hop	0.303	0.232 to 0.376	0.184	0.328	0.354	0.399
embedding_seed	0.202	0.149 to 0.260	0.091	0.263	0.336	0.432
hybrid_rrf	0.377	0.310 to 0.446	0.220	0.446	0.574	0.779
hybrid_bm25_graph	0.386	0.318 to 0.455	0.216	0.457	0.582	0.784

BM25 is the strongest single method, and the two hybrids are nominally the best overall. The intervals overlap heavily.

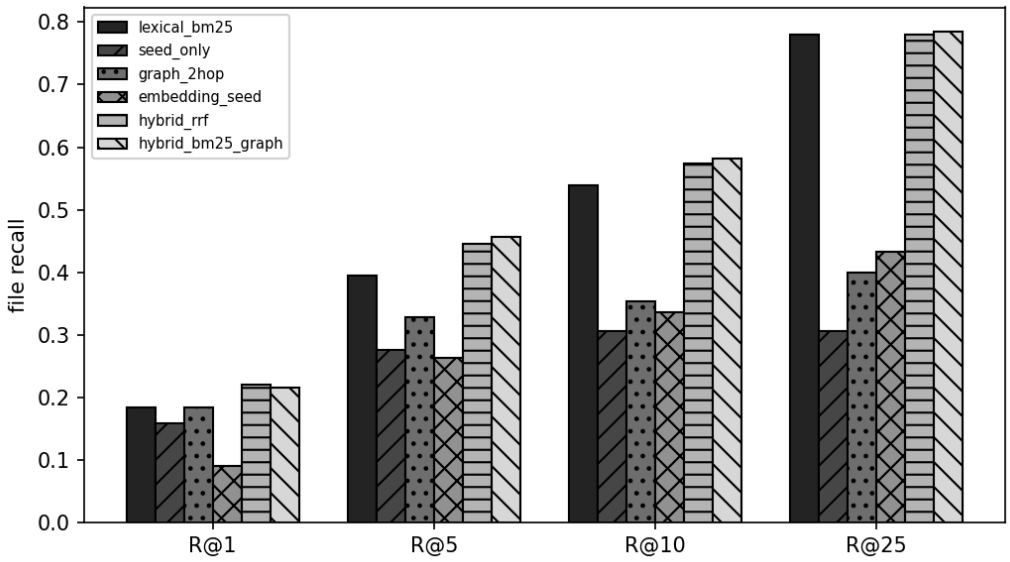


Figure 1. File recall at k for each retrieval signal (124 tasks).

4.2 Paired comparisons

Comparison	Mean MRR difference	95% CI
hybrid_bm25_graph minus lexical_bm25	+0.026	-0.034 to +0.086
hybrid_rrf minus lexical_bm25	+0.017	-0.046 to +0.081
graph_2hop minus seed_only	+0.049	+0.017 to +0.082
embedding_seed minus graph_2hop	-0.100	-0.167 to -0.033
hybrid_bm25_graph minus hybrid_rrf	+0.009	-0.029 to +0.046

Three findings are supported by intervals that exclude zero or that clearly include it. First, expanding the seed through the call graph helps, plausibly because callers and callees of the named symbol are often edited together. Second, replacing the graph with embedding similarity hurts, so the stored embeddings add less structure than the call edges do. Third, fusing signals with BM25 does not give a demonstrated gain over BM25 alone, and we do not claim one.

4.3 The best signal depends on the repository

Repository	n	BM25	seed_only	graph_2hop	embedding_seed	hybrid_rrf	hybrid_bm25_graph
FastAPI	64	0.389	0.150	0.218	0.151	0.343	0.340
Rich	46	0.218	0.355	0.347	0.189	0.336	0.381
requests	13	0.666	0.387	0.511	0.442	0.645	0.579

On FastAPI, BM25 is far ahead of every seeded method. On Rich, the seed alone (0.355) and the graph (0.347) beat BM25 (0.218), and the fusion of BM25 and graph (0.381) is best. We offer a hypothesis and do not test it here: Rich issues tend to name the class or function that is wrong, which gives seeds a high hit rate, whereas many FastAPI issues describe behavior in prose, or concern documentation sources, where text search does the work. A robust agent should therefore use text search first and graph tools when the issue names a symbol.

4.4 Seeded tasks only

On the 90 tasks with a seed, BM25 reaches an MRR of 0.435, graph expansion 0.417, embedding similarity 0.279, the fusion of all three 0.459, and the fusion of BM25 and graph 0.471. The ordering matches the full set, and the gap between the graph and BM25 narrows when a seed exists.

4.5 Token cost

The estimated cost of returning the source text of the top ten nodes is about 15,500 tokens for BM25, 10,700 for graph expansion, 6,800 for the seed alone, 3,300 for embedding similarity, and about 16,000 to 19,600 for the hybrids. With a window of 32,768 tokens that must also hold the system prompt, the issue, tool schemas, code that is read, and the patch, a tool that returns ten full nodes can consume nearly half of the budget. The competition's tools cap command and graph output at 5,000 characters, which bounds the worst case.

4.6 Does choosing the signal per issue help?

We test the idea with two routers. The rule router uses the BM25 and graph fusion when a seed exists and plain BM25 otherwise. It reaches an MRR of 0.386 (95% CI 0.318 to 0.455), which is identical to the fixed fusion, because without a seed the graph contributes nothing and the fusion reduces to BM25. An oracle router that picks the best of all six methods for each task, using the gold labels, reaches 0.544 (0.473 to 0.616), and an oracle that picks only between BM25 and the fusion reaches 0.465 (0.395 to 0.535). These oracle numbers are upper bounds that no deployed router can reach, but they show that the signals fail on different tasks. A seed-existence rule is not enough.

4.7 Sensitivity and learned routing

The conclusions do not depend on our settings. Graph expansion gives an MRR of 0.296, 0.303, and 0.316 for one, two, and three hops, and the BM25 and graph fusion gives 0.398, 0.386, and 0.389 for fusion constants of 10, 60, and 100, with overlapping intervals throughout. We also trained a logistic regression router on issue features (length, seed count, identifier count, traceback presence, code spans, and repository) to choose between BM25 and the fusion, evaluated with leave-one-out validation. With the repository as a feature it reaches 0.387, a difference of +0.001 from the fixed fusion (interval -0.052 to +0.053). Without the repository feature it reaches 0.359, no better than

BM25. A router trained on 124 tasks therefore does not capture the oracle headroom, and we leave larger training sets and model-based routing to future work.

5. Discussion

What a prompt author should do. The evidence supports a simple policy for small agents. Start with text search over the issue's identifiers, because it is the strongest and the most robust single signal. When the issue names a symbol that exists in the graph, call the neighbor tool on that symbol, because graph expansion beats the bare seed with a clear margin and costs less context than ten BM25 hits. Treat embedding similarity as a last resort. We caution that an agent prompt following this policy scored lower end to end (Section 6), so the policy is a statement about retrieval quality and not a demonstrated improvement of the agent.

Why aggregate gains are small. The hybrids rescue some tasks that BM25 misses and demote others that BM25 gets right, and the repository breakdown shows that these effects have opposite signs in different repositories. A fixed fusion rule averages them out. Our simple and learned routers did not help, but the oracle gap in Section 4.6 shows that an agent that chooses its signal after reading the issue could in principle gain a lot, and we regard this as the most promising direction.

6. Limitations

We state the limits of this study plainly.

- The benchmark covers 124 tasks from four repositories, and FastAPI and Rich account for 110 of them. The per-repository differences are descriptive and would need more tasks for confirmation.
- Files are matched through module paths and not through the repository snapshots, so the match can be wrong for unusual layouts.
- Seeds come from exact name matching in the issue text. A language model may pick better or worse seeds, so our seeded results are an approximation of agent behavior.
- We measure retrieval quality only. We did not run a language model inside the study. For context, our baseline agent configuration scored 0.05 on the public leaderboard, which shows that localization is only one of the factors that decide the resolution rate, and a later configuration whose prompt follows the retrieval policy of Section 5 scored 0.01, lower than the baseline. The retrieval differences therefore did not translate into a better resolution rate in our setting, although one leaderboard score on a small task set is a noisy measure and the two configurations also differ in other prompt rules.
- The tasks are the public training split, and no result here applies to the hidden test tasks.
- Token costs are estimates based on character counts.

7. Reproducibility

All code is in a public repository at https://github.com/Blackl1stV35/gemma4-agent-localization. The study needs only the competition's task file, the graph files, and the embedding files, plus Python with NumPy and Matplotlib. The scripts `paper/study.py`, `paper/analysis.py`, `paper/router.py`, and `paper/extra.py` write the results, intervals, router results, and sensitivity tables. All scripts run

on a CPU in under a few minutes. The `.md` and `.json` files next to them contain every reported number.

8. Conclusion

For small offline agents with a tight context budget, plain lexical search is a strong localization baseline, call-graph expansion from a named symbol improves on the bare symbol, and stored embeddings are the weakest of the signals we tested. Combining signals does not significantly improve on lexical search in aggregate, and the best signal depends on the repository. We release a small, exactly reproducible benchmark so that future submissions can test retrieval policies before they spend a model run.

References

- LocAgent: Graph-Guided LLM Agents for Code Localization (2025). arXiv:2503.09089.
- Cormack, G. V., Clarke, C. L. A., and Büttcher, S. (2009). Reciprocal Rank Fusion outperforms Condorcet and individual rank learning methods. SIGIR 2009, 758-759.
- Issue Localization via LLM-Driven Iterative Code Graph Searching, CoSIL (2025). arXiv:2503.22424.
- Jimenez, C. E., et al. (2024). SWE-bench: Can Language Models Resolve Real-World GitHub Issues? ICLR 2024. arXiv:2310.06770.
- Ouyang, S., et al. (2025). RepoGraph: Enhancing AI Software Engineering with Repository-level Code Graph. ICLR 2025. arXiv:2410.14684.
- Robertson, S., and Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. Foundations and Trends in Information Retrieval 3(4), 333-389.
- Xia, C. S., Deng, Y., Dunn, S., and Zhang, L. (2024). Agentless: Demystifying LLM-based Software Engineering Agents. arXiv:2407.01489.
- Yang, J., et al. (2024). SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. NeurIPS 2024. arXiv:2405.15793.